UNIVERSITY OF INFORMATION TECHNOLOGY
FACULTY OF INFORMATION SYSTEMS

Chapter 6

FUNCTIONAL DEPENDENCIES AND NORMALIZATION FOR RELATIONAL DATABASES

Dr. Cao Thi Nhan

Content

1. Informal Design Guidelines for Relation Schemas
2. Functional Dependencies
3. Normal Forms Based on Primary Keys
4. General Definitions of Second and Third Normal Forms
5. Boyce-Codd Normal Form

Informal Design Guidelines for Relation Schemas

- Levels at which we can discuss goodness of relation schemas
  - Logical (or conceptual) level
  - Implementation (or physical storage) level
- Measures of quality
  - Making sure attribute semantics are clear
  - Reducing redundant information in tuples
  - Reducing NULL values in tuples
  - Disallowing possibility of generating spurious tuples

Informal Design Guidelines for Relation Schemas

- Imparting Clear Semantics to Attributes in Relations
  - Semantics of a relation: Meaning resulting from interpretation of attribute values in a tuple
  - Easier to explain semantics of relation: Indicates better schema design

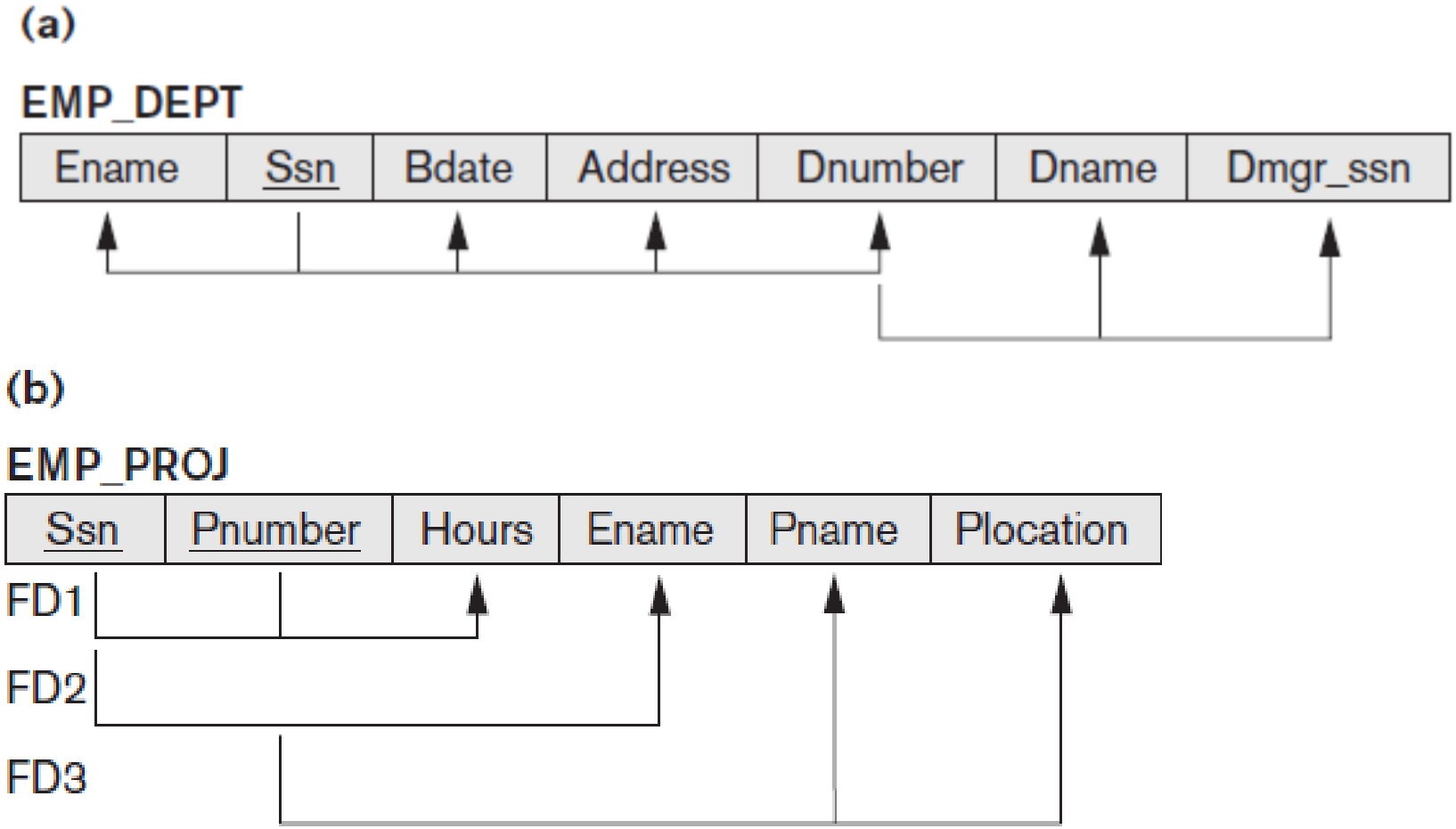
Guideline 1

- Design relation schema so that it is easy to explain its meaning
- Do not combine attributes from multiple entity types and relationship types into a single relation
- Example of violating Guideline 1: Figure 15.3

Guideline 1

- Do not combine attributes from multiple entity types and relationship types into a single relation

Figure 15.3
Two relation schemas suffering from update anomalies. (a) EMP_DEPT and (b) EMP_PROJ.



Redundant Information in Tuples and Update Anomalies

- Grouping attributes into relation schemas
  - Significant effect on storage space
- Storing natural joins of base relations leads to **update anomalies**
- Types of update anomalies:
  - Insertion
  - Deletion
  - Modification

Guideline 2

- Design base relation schemas so that no update anomalies are present in the relations
- If any anomalies are present:
  - Note them clearly
  - Make sure that the programs that update the database will operate correctly

NULL Values in Tuples

- May group many attributes together into a “fat” relation
  - Can end up with many NULLs
- Problems with NULLs
  - Wasted storage space
  - Problems understanding meaning

Guideline 3

- Avoid placing attributes in a base relation whose values may frequently be NULL
- If NULLs are unavoidable:
  - Make sure that they apply in exceptional cases only, not to a majority of tuples

NULL Values in Tuples

- May group many attributes together into a “fat” relation
  - Can end up with many NULLs
- Problems with NULLs
  - Wasted storage space
  - Problems understanding meaning

Generation of Spurious Tuples

- Figure 14.5(a)
  - Relation schemas EMP_LOCS and EMP_PROJ1
- NATURAL JOIN
  - Result produces many more tuples than the original set of tuples in EMP_PROJ
  - Called **spurious tuples**
  - Represent spurious information that is not valid

Generation of Spurious Tuples

EMP_LOCS

Ename	Plocation

P.K.

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation

P.K.

EMP_LOCS

Ename	Plocation
Smith, John B.	Bellaire
Smith, John B.	Sugarland
Narayan, Ramesh K.	Houston
English, Joyce A.	Bellaire
English, Joyce A.	Sugarland
Wong, Franklin T.	Sugarland
Wong, Franklin T.	Houston
Wong, Franklin T.	Stafford
Zelaya, Alicia J.	Stafford
Jabbar, Ahmad V.	Stafford
Wallace, Jennifer S.	Stafford
Wallace, Jennifer S.	Houston
Borg, James E.	Houston

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation
123456789	1	32.5	ProductX	Bellaire
123456789	2	7.5	ProductY	Sugarland
666884444	3	40.0	ProductZ	Houston
453453453	1	20.0	ProductX	Bellaire
453453453	2	20.0	ProductY	Sugarland
333445555	2	10.0	ProductY	Sugarland
333445555	3	10.0	ProductZ	Houston
333445555	10	10.0	Computerization	Stafford
333445555	20	10.0	Reorganization	Houston
999887777	30	30.0	Newbenefits	Stafford
999887777	10	10.0	Computerization	Stafford
987987987	10	35.0	Computerization	Stafford
987987987	30	5.0	Newbenefits	Stafford
987654321	30	20.0	Newbenefits	Stafford
987654321	20	15.0	Reorganization	Houston
[illegible]	[illegible]	NULL	Reorganization	Houston

Generation of Spurious Tuples

EMP_LOCS

Ename	Plocation

P.K.

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation

P.K.

	Ssn	Pnumber	Hours	Pname	Plocation	Ename
	123456789	1	32.5	ProductX	Bellaire	Smith, John B.
*	123456789	1	32.5	ProductX	Bellaire	English, Joyce A.
	123456789	2	7.5	ProductY	Sugarland	Smith, John B.
*	123456789	2	7.5	ProductY	Sugarland	English, Joyce A.
*	123456789	2	7.5	ProductY	Sugarland	Wong, Franklin T.
	666884444	3	40.0	ProductZ	Houston	Narayan, Ramesh K.
*	666884444	3	40.0	ProductZ	Houston	Wong, Franklin T.
*	453453453	1	20.0	ProductX	Bellaire	Smith, John B.
	453453453	1	20.0	ProductX	Bellaire	English, Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Smith, John B.
	453453453	2	20.0	ProductY	Sugarland	English, Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Wong, Franklin T.

Guideline 4

- Design relation schemas to be joined with equality conditions on attributes that are appropriately related
  - Guarantees that no spurious tuples are generated
- Avoid relations that contain matching attributes that are not (foreign key, primary key) combinations

Summary and Discussion of Design Guidelines

- Anomalies cause redundant work to be done
- Waste of storage space due to NULLs
- Difficulty of performing operations and joins due to NULL values
- Generation of invalid and spurious data during joins

Group Discussion

- Time: 15 minutes.
- These relations are good? Why? Your improvements?

SUPPLIER_DETAIL (S_ID, Address, Product_ID, Price)

S_ID	Address	Product_ID	Price
NCC01	10 Lê Duẩn, Q.1	MH01	5.000
NCC01	10 Lê Duẩn, Q.1	MH02	25.000
NCC01	10 Lê Duẩn, Q.1	MH03	6.000
NCC02	25 Hoàng Sa, Q.3	MH01	4.500
NCC02	25 Hoàng Sa, Q.3	MH05	35.500

Group Discussion

- Time: 05 minutes.
- These relations are good? Why?

SUPPLIER_DETAIL (S_ID, Address, Product_ID, Price)

S_ID	Address	Product_ID	Price
NCC01	10 Lê Duẩn, Q.1	MH01	5.000
NCC01	10 Lê Duẩn, Q.1	MH02	25.000
NCC01	10 Lê Duẩn, Q.1	MH03	6.000
NCC02	25 Hoàng Sa, Q.3	MH01	4.500
NCC02	25 Hoàng Sa, Q.3	MH05	35.500

Group Discussion

SUPPLIER_DETAIL (S_ID, Address, Product_ID, Price)

Insert a new Supplier?

S_ID	Address	Product_ID	Price
NCC01	10 Lê Duẩn, Q.1	MH01	5.000
NCC01	10 Lê Duẩn, Q.1	MH02	25.000
NCC01	10 Lê Duẩn, Q.1	MH03	6.000
NCC02	25 Hoàng Sa, Q.3	MH01	4.500
NCC02	25 Hoàng Sa, Q.3	MH05	35.500
NCC03	1 Nguyễn Văn Trỗi, Q.3	?	?

Group Discussion

SUPPLIER_DETAIL (S_ID, Address, Product_ID, Price)

Delete Produst_ID and its price?

S_ID	Address	Product_ID	Price
NCC01	10 Lê Duẩn, Q.1	MH01	5.000
NCC01	10 Lê Duẩn, Q.1	MH02	25.000
NCC01	10 Lê Duẩn, Q.1	MH03	6.000
NCC02	25 Hoàng Sa, Q.3	MH01	4.500
NCC02	25 Hoàng Sa, Q.3	MH05	35.500

Group Discussion

SUPPLIER_DETAIL (S_ID, Address, Product_ID, Price)

2 relations:

- SUPPLIER (S_ID, Address)
- SUPPLIER_INFOR (S_ID, Product_ID, Price)

Content

1. Informal Design Guidelines for Relation Schemas
2. Functional Dependencies
3. Normal Forms Based on Primary Keys
4. General Definitions of Second and Third Normal Forms
5. Boyce-Codd Normal Form

Functional Dependencies

- Formal tool for analysis of relational schemas
- Enables us to detect and describe some of the above-mentioned problems in precise terms
- Theory of functional dependency

Definition of Functional Dependency

- Constraint between two sets of attributes from the database

Definition. A **functional dependency**, denoted by $X \rightarrow Y$, between two sets of attributes X and Y that are subsets of R specifies a *constraint* on the possible tuples that can form a relation state r of R. The constraint is that, for any two tuples t_1 and t_2 in r that have $t_1[X] = t_2[X]$, they must also have $t_1[Y] = t_2[Y]$.

- Property of semantics or meaning of the attributes
- **Legal relation states**
  - Satisfy the functional dependency constraints

Definition of Functional Dependency

- Given a populated relation
  - Cannot determine which FDs hold and which do not
  - Unless meaning of and relationships among attributes known
  - Can state that FD does not hold if there are tuples that show violation of such an FD

Definition of Functional Dependency

- Example: OrderDetail (OrID, Date, Emp_ID, Cust_ID, Total, ProductID, Quantity, UnitPrice)
- Functional Dependencies:
  - OrID, ProductID → Date, Emp_ID, Cust_ID, Total, Quantity, UnitPrice
  - OrID → Date, Emp_ID, Cust_ID, Total
  - ProductID → UnitPrice
- ???
  - OrID, ProductID → Date, Emp_ID, Cust_ID, UnitPrice
  - OrID → Date, Emp_ID, Cust_ID, Total
  - ProductID → UnitPrice

Group Discussion

- OrderDetail (OrID, ProductID, Quantity, UnitPrice, Total)
- Functional Dependencies:
  - OrID, ProductID → Quantity, UnitPrice, Total
  - Quantity, UnitPrice → Total

The F1= {Quantity, UnitPrice → Total} can be the same as

F2= {Quantity → Total; UnitPrice → Total} ?

Normalization of Relations

- Takes a relation schema through a series of tests
  - Certify whether it satisfies a certain normal form
  - Proceeds in a top-down fashion
- Normal form tests

Definition. The **normal form** of a relation refers to the highest normal form condition that it meets, and hence indicates the degree to which it has been normalized.

Normalization of Relations

- Properties that the relational schemas should have:
  - **Nonadditive join property**
    - ✓ Extremely critical
  - **Dependency preservation property**
    - ✓ Desirable but sometimes sacrificed for other factors

TenNhaCC	DiaChi
Co.opmart	127 Đinh Tiên Hoàng, Q.1
Co.opmart	48 Hoa Sứ, Phú Nhuận

JOIN

TenNhaCC	MaHang	Gia
Co.opmart	MH01	5.000
Co.opmart	MH02	25.000
Co.opmart	MH03	6.000
Co.opmart	MH01	4.500
Co.opmart	MH05	35.500

TenNhaCC	DiaChi	MaHang	Gia
Co.opmart	127 Đinh Tiên Hoàng, Q.1	MH01	5.000
Co.opmart	127 Đinh Tiên Hoàng, Q.1	MH02	25.000
Co.opmart	127 Đinh Tiên Hoàng, Q.1	MH03	6.000
Co.opmart	127 Đinh Tiên Hoàng, Q.1	MH01	4.500
Co.opmart	127 Đinh Tiên Hoàng, Q.1	MH05	35.500
Co.opmart	48 Hoa Sứ, Phú Nhuận	MH01	5.000
Co.opmart	48 Hoa Sứ, Phú Nhuận	MH02	25.000
Co.opmart	48 Hoa Sứ, Phú Nhuận	MH03	6.000
Co.opmart	48 Hoa Sứ, Phú Nhuận	MH01	4.500
Co.opmart	48 Hoa Sứ, Phú Nhuận	MH05	35.500

Normalization of Relations

Compare 2 situations:

S1:

- OrderDetail (<u>OrID, ProductID</u>, Quantity, UnitPrice, Total)
- Product (<u>ProductID</u>, Name)

And

S2:

- OrderDetail (<u>OrID, ProductID</u>, Quantity, Total)
- Product (<u>ProductID</u>, Name, UnitPrice)

Practical Use of Normal Forms

- Normalization carried out in practice
  - Resulting designs are of high quality and meet the desirable properties stated previously
  - Pays particular attention to normalization only up to 3NF, BCNF, or at most 4NF
- Do not need to normalize to the highest possible normal form

Definition. Denormalization is the process of storing the join of higher normal form relations as a base relation, which is in a lower normal form.

Definitions of Keys and Attributes Participating in Keys

- Definition of **superkey** and **key**
- **Candidate key**
  - If more than one key in a relation schema
    - ✓ One is **primary key**
    - ✓ Others are **secondary keys**

Definition. An attribute of relation schema R is called a **prime attribute** of R if it is a member of *some candidate key* of R. An attribute is called **nonprime** if it is not a prime attribute—that is, if it is not a member of any candidate key.

First Normal Form

- Only attribute values permitted are single atomic (or indivisible) values
- Techniques to achieve first normal form
  - Remove attribute and place in separate relation
  - Expand the key
  - Use several atomic attributes

First Normal Form

- CTietMua is not single atomic value

MaHD	MaKH	NgayHD	CtietMua	SoTien
HD01	KH01	15-10-15	Bánh Orion, 1, gói Kẹo mút Chupa Chups, 2, cây	25.000
HD02	KH01	18-10-15	Gạo, 2, kg	30.000
HD03	KH02	24-10-15	Đường, 1, kg Bánh AFC, 2, gói	39.000

- Hdon (MaHD, MaKH, NgauHD, SoTien)
- CTietHD (MaHD, MaSP, SL)

First Normal Form

- Does not allow **nested relations**
  - Each tuple can have a relation within it
- To change to 1NF:
  - Remove nested relation attributes into a new relation
  - Propagate the primary key into it
  - **Unnest** relation into a set of 1NF relations

First Normal Form

- Does not allow **nested relations**
  - Each tuple can have a relation within it
- To change to 1NF:
  - Remove nested relation attributes into a new relation
  - Propagate the primary key into it
  - **Unnest** relation into a set of 1NF relations

(a)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations

(b)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Figure 15.9
Normalization into 1NF. (a) A relation schema that is not in 1NF. (b) Sample state of relation DEPARTMENT. (c) 1NF version of the same relation with redundancy.

Second Normal Form

- Based on concept of **full functional dependency**
  - Versus **partial dependency**

Definition. A relation schema R is in 2NF if every nonprime attribute A in R is *fully functionally dependent* on the primary key of R.

- Second normalize into a number of 2NF relations
  - Nonprime attributes are associated only with part of primary key on which they are fully functionally dependent

Second Normal Form

- Based on concept of **full functional dependency**
  - Versus **partial dependency**

Definition. A relation schema R is in 2NF if every nonprime attribute A in R is *fully functionally dependent* on the primary key of R.

- Second normalize into a number of 2NF relations
  - Nonprime attributes are associated only with part of primary key on which they are fully functionally dependent

Second Normal Form

- Suppose STUDENT (StudentID, CourseID, Name, Address, Score)
- F={StudentID, CourseID → Score; StudentID→Name, Address}
- STUDENT is not in 2NF because:
  1. Key: {StudentID, CourseID}, Nonprime Attribute: {Name, Address, Score}
  2. StudentID, CourseID → Score → OK
  3. StudentID→Name, Address: Name, Address are nonprime attribute and depend on a part of key → not in 2NF
- Design 2 relations
- Partticipate (StudentID, CourseID, Score); F1={StudentID, CourseID → Score}
- Student (StudentID, Name, Address); F2={StudentID→Name, Address}

Third Normal Form

- Based on concept of transitive dependency

Definition. According to Codd's original definition, a relation schema R is in 3NF if it satisfies 2NF *and* no nonprime attribute of R is transitively dependent on the primary key.

Definition. A relation schema R is in **third normal form** (3NF) if, whenever a *nontrivial* functional dependency $X \rightarrow A$ holds in R, either (a) X is a superkey of R, or (b) A is a prime attribute of R.

- All FDs X→A:
  - Left side includes a key (or a superkey), Or
  - Right side is a key attribute

BC Normal Form

- All FDs X→A:
  - Left side includes a key (superkey)

Definition. A relation schema R is in **BCNF** if whenever a *nontrivial* functional dependency $X \rightarrow A$ holds in R, then X is a superkey of R.

Inference Rules for Functional Dependencies

- X→Y: Funtional dependency
- F: Functional dependency set
- Dependencies can be inferred or deduced from the FDs in F.
- Closure of F, is denoted by F+: set of all dependencies that include F as well as all dependencies that can be inferred from F

Question: How to show that X→Y is a member of F? (Or X→Y ∈ F+)

1. Armstrong's axioms
2. X→Y is a member of F ⇔ $Y \subseteq X_F^+$

- X_F^+: the closure of X with respect to F. *Question: How to determine all the attributes that can be functionally determined from X?*

Inference Rules, Armstrong's axioms

Armstrong's axioms.[1]

IR1 (reflexive rule)[2]: If $X \supseteq Y$, then $X \rightarrow Y$.

IR2 (augmentation rule)[3]: $\{X \rightarrow Y\} \models XZ \rightarrow YZ$.

IR3 (transitive rule): $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$.

IR4 (decomposition, or projective, rule): $\{X \rightarrow YZ\} \models X \rightarrow Y$.

IR5 (union, or additive, rule): $\{X \rightarrow Y, X \rightarrow Z\} \models X \rightarrow YZ$.

IR6 (pseudotransitive rule): $\{X \rightarrow Y, WY \rightarrow Z\} \models WX \rightarrow Z$.

$F \models X \rightarrow Y$ to denote that the functional dependency $X \rightarrow Y$ is inferred from F

Inference Rules

Question: How to show that X→Y is a member of F? (Or X→Y ∈ F+)

- X→Y is a member of F ⇔ $Y \subseteq X_F^+$
- X_F^+: the closure of X with respect to F. *Question: How to determine all the attributes that can be functionally determined from X?*

Algorithm 15.1. Determining X^+, the Closure of X under F

Input: A set F of FDs on a relation schema R, and a set of attributes X, which is a subset of R.

$X^+ := X$;
repeat
    $\text{old}X^+ := X^+$;
    for each functional dependency $Y \rightarrow Z$ in F do
        if $X^+ \supseteq Y$ then $X^+ := X^+ \cup Z$;
    until $(X^+ = \text{old}X^+)$;

Inference Rules

Given R(A,B,C,D,E,G,H) and

F={f1: B → A, f2: DA → CE, f3: D → H, f4: GH → C, f5: AC → D}

Find AC^+_F

Step 1: $AC^+_F = AC$

Step 2: loop for each FD

$AC^+_F = AC \cup D = ACD$ (because f5)

$AC^+_F = ACD \cup CE = ACDE$ (because f2)

$AC^+_F = ACDE \cup H = ACDEH$ (because f3)

$AC^+_F = ACDEH$ (cannot add any attribute)

Result: AC^+_F=ACDEH

Inference Rules

Given R(A,B,C,D,E,G,H) and

F={f1: B → A, f2: DA → CE, f3: D → H, f4: GH → C, f5: AC → D}

Question 1: AC→E ∈ F⁺? X→Y is a member of F ⇔ $Y \subseteq X_F^+$

Step 1: AC^+_F = ACDEH

Step 2: Because E ⊆ AC^+_F, so AC→E ∈ F⁺

Question 2: Prove that AC→ B is NOT a member of F.

Step 1: AC^+_F = ACDEH

Step 2: Because B ⊈ AC^+_F, so AC→B ∉ F⁺

Group Discussion – 10 minutes

Given R(A, B, C, D, E, G, H) and

F={f1: AH → BC, f2: B → E, f3: H → A, f4: EG → H, f5: G → D}

Prove that BG→ A ∈ F+

Finding all key of a relation

Input: R and F; Output: Set of key K

1. Step 1: Find S: set of attribute that only include in the left side of FDs
2. Step 2: If $S^+_F = R^+$, R has only one key is T; otherwise go to step 3
3. Step 3: Find all $Ti \subseteq T$ ($Ti \neq \emptyset$), which T is the set of attribute that both include in left side of FDs and right side of FDs.
4. Step 4: For each Ti, if $(S \cup Ti)^+_F = R^+$ then $Ki = (S \cup Ti)$, ignore all Tj: $Ti \subset Tj$
5. Step 5: Conclude K={Ki}

Finding all key of a relation

Given R(A, B, C, D, E, G, H) and F={B→A, DA→CE, D→H, GH→C, AC→D}

- Step 1: S=BG; Step 2: $S^+_F = BGA^+ \neq R^+$, go to step 3
- Step 3: T=ACDH, set of all Ti ⊆ T (Ti ≠ ∅): {A, C, D, H, AC, AD, AH, CD, CH, DH, ACD, ACH, ADH, CDH, ACDH}
- Step 4: For each Ti, if $(S \cup Ti)^+_F = R^+$ then Ki=(S∪Ti), ignore all Tj: Ti⊂Tj

S∪Ti	$(S \cup Ti)^+_F$	$(S \cup Ti)^+_F = R^+$	Explanation
BGA	BGA	False	
BGC	BGCADEH	True	BGC is a key, igrore check on all Ti contained C: AC, CD, CH, ACD, ACH, CDH, ACDH
BGD	BGDACEH	True	BGD is a key, igrore check on all Ti contained D: CD, DH, ADH
BGH	BGHACDE	True	BGH is a key.

- Step 5: Conclude K={BGC, BGD, BGH}

Group Discussion – 10 minutes

Given R(A, B, C, D, E, G, H) and

F={f1: AH → BC, f2: B → E, f3: H → A, f4: EG → H, f5: G → D}

Find all keys of (R, F)

Group Discussion – 10 minutes

Given R(A, B, C, D, E, G, H) and

F={f1: AH → BC, f2: B → E, f3: H → A, f4: EG → H, f5: G → D}

Is (R, F) qualified for 2nd NF? Why?

Group Discussion – 10 minutes

Given R(A, B, C, D, E, G, H) and

F={f1: AH → BC, f2: B → E, f3: H → A, f4: EG → H, f5: G → D}

Is (R, F) qualified for 3nd NF? Why?

Q A

References

1. Ramez Elmasri, and Shamkant B. Navathe, *Fundamentals of Database Systems (7th Ed.)*, Addison-Wesley, 2016.

2. Nguyen Dang Ty, and Do Phuc, *Database*, Vietnam National University – HCMC, 2015.

3. George Tillmann, *Usage-Driven Database Design,* Apress, 2017.